

Date oop

- composition
- association
- aggregation

- inheritance
→ polymorphism

Classes & Objects

```
class Rectangle {  
    int width;  
    int length;  
};
```

could be accessed.

class Rectangle {

private:

```
    int width;  
    int length;
```

so can't be access in main.

could be accessed.

public:

```
    void setheight (int h)  
    { height = h;  
    }
```

```
    void setwidth (int w)  
    { width = w;  
    }
```

```
    int getwidth ()
```

```
    { return width;  
    }
```

```
};
```

* methods / functions

* access modifiers
private public

```
Rectangle r;  
r.setheight(5);
```

* By default class is private.
↳ information hiding

* Encapsulation → encapsulate data & functionalities

* Setter / Getter functions.]

* Accessor and mutators
Getter Setter

Set & Get private data values.

Better to make setter & Getter separate for each member.

if we are defining methods outside of class we have to use scope resolution operator.

```
class Student {
```

```
    private:
```

```
    int roll;
```

```
    public:
```

```
    void setroll (int r);  
};
```

```
void Student::setroll (int r)  
{  
    roll = r;  
}
```

Date

Constructors: → No return type (not even void)

→ Are functions

→ It's the class name

→ If no parameters are passed, it is default constructor.

→ we can write more than 1 constructor, & is called constructor Overloading.

```
class Algebra {
```

```
private
```

```
int a;
```

```
int b;
```

```
public:
```

```
Algebra () { → constructor. (default)
```

```
    a = 0;  
    b = 0;  
}
```

```
Algebra (int a) → constructor
```

```
{
```

```
    a = b = 1; }  
Algebra (int i, int j) { → parameterized constructor
```

```
{
```

```
    a = i;    b = j; }
```

Algebra (int i) { a(i), b(i) } → would

also do

this

& is called

member initializer list.

→ also exists as default.

→ Copy constructor → receive a object of that class & creates a copy.

```
Algebra o1 (05, 10); → Algebra o2 = o1; → can also do this.
```

```
Algebra o2 (o1); → copy constructor.
```

```
Algebra (const Algebra &obj) { a(obj.a), b(obj.b)
```

*if in copy constructor we don't pass by reference, it will call an error.

```
~Algebra () → Destructor.
```

Date

Date * P;

Date d(5, 3, 1994);

P: Adj; → store address of day/date.

(*P).display(); // precedence of * is higher than (),

P → display();

Date arr[5] = { Date(1, 9, 1995), Date - - - };

Date[i].display;

(*(&arr[i])).display; *Date * arr = new Date[5] { Date(1, 2, 1999) ~ };

→ Destructor:

- Default.

- ~ Date();

or used when an obj. lifetime ends.

4th Return type.

}

A Destructor can't be overload, only one.

- Destructor is called in Reverse order of constructor.

Date O1;

Date O2(09, 10, 1997); → Destructor will be called first as its constructor was called second.

- For Dynamic memory, you have to call delete(); only then the destructor will be called.

- Could be called with the object, but will also be called with the default. O1. ~delete();

- member is initialized in member initialization list.

Algebra (inta; intb; intc): {c} // is const.

Date

```
class Set {
```

```
    private:
```

```
        int * data;
```

```
        int capacity;
```

```
        int noOfElements;
```

```
    public:
```

```
        Set() {
```

```
            capacity = 10;
```

```
            noOfElements = 0;
```

```
            data = new int[capacity];
```

```
        }
```

```
        Set(const Set &ref)
```

```
        { capacity = ref.capacity;
```

```
          noOfElements = ref.noOfElements;
```

```
          data = new int[capacity];
```

```
          for (int i = 0; i < noOfElements; i++)
```

```
              { data[i] = ref.data[i];
```

```
                }
```

```
          }
```

```
        ~Set() {
```

```
            delete [] data;
```

```
            data = nullptr;
```

```
        }
```

→ Shallow Copy: exactly copy all variables, By default.

But does not work for Dynamic Array. As it will copy address not create another copy. Does not work for pointers too.

→ Bcz of shallow copy, we have to write our own Copy constructor when elements have dynamic array or pointers, & copy all elements or values. This Copy constructor is "Deep Copy".

→ We also have to write destructor when there is dynamic mem.

→ Copy constructor is called:

- New object
- Pass by value
- Return by value

Date

- * Const object can only call const function.
- * We can write two function of same name & parameters and write const with one not with other.
- * function overloading.

* this pointer:
as hidden parameter with every member of class.
address of calling object.

this → len = len;
*(this).len = len;

* Just write prototype of function & constructor in class & define later on.

void Rectangle::setvalue(int a, int b);

}*

Do this with function when defining outside class.
Rectangle::Rectangle() { otherwise will be global.

}
→ With constructor*

obj.print1(), print2(); → we can do this if first func don't have void.

* Chaining → chain function calling

Date 11-03-20

Static function & Variable

class A {

private:

static int a;

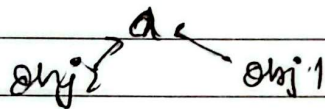
int x;

public:

static void print() {

x = 1; } → can't work

- * All objects of class will be sharing the static variable. It will be only one.



no need for object or instance to call static functions.

TO call we

simplify write class name scope resolution operator & function

* A::print();

But we can create object.

- * Static member function
- Can't access instance data member

```
Static int getX() {
    return a;
}
```

* after declaring class we have to declare or initialize the static member.

cout << A::getX();

* int A::a = 10;

* we can't use "this" in static functions.

* ~~non~~ static can't access non-static members.

* non-static can access static.

* Static Array

* can only be made inside class.

* members are made on global data section.

class CString {

int size;

char * data; }

strlen
strcpy
strcmp
strcat
strncpy

Date

```
class MyArray {  
    private:  
        int *data;  
        int size;
```

```
main() { MyArray a1;  
        MyArray a3(a2); → deep copy  
        a1 = a3 → shallow copy
```

```
void copy (MyArray ref) {  
    this → ~MyArray();  
    this → size = ref.size  
    this → data = new int [size]  
    for (int i=0; i<size; i++)  
        del[i] = ref.data[i];  
}
```

Operator Overloading:

* void operator $\Rightarrow$ (const MyArray &ref) → By this a1 = a3 will work.
{ * assignment operator.

} if Return is void, we can't do this a1 = a2 = a3
so we change return type to MyArray.

MyArray & operator = (const MyArray &ref) {

return *this; }

~~int~~
MyArray operator [] (int index) { * index operator
return p[index];

a1[a2] = 11;

MyArray operator + (const MyArray &ref) { * add operator
if (this → size == ref.size) {
 MyArray newObj (this → ref);
 for (int i=0; i<size; i++)
 newObj[i] = data[i] + ref.data[i];
 return newObj; }
a3 = a1 + a2;

Date

bool

```
bool operator==(const myArray& ref){  
    if (this->size == ref.size){  
        for (int i=0; i<size; i++){  
            if (at[i] == ref.at[i]){  
                return true;  
            }  
        }  
        return false;  
    }  
}
```

+= -= *= /= > < >= <= +

return *this;

i++ ++i → NO parameter needed. ← i-- --i
pre post

```
point& operator--(){ → pre decrement ← ++()  
    x--;  
    y--;  
    return *this;  
}
```

```
point& operator--(int){ → post decrement ← ++(int)  
    point temp = *this;  
    x--;  
    y--;  
    return temp;  
}
```

```
operator double(){ → to convert a object to double.  
    return (x+y)*0.001;  
}
```

```
operator int(){  
    return (x+y);  
}
```

Date * Friend Classes & Functions

class A;

```
private:
    int a;
    void getA();
}
```

class B;

can't be accessed as private data member.

```
void g(A obj) {
    cout << obj.a;
}
```

```
int main() {
    A oa(10);
    B ob(100);
    ob.g(oa);
}
```

class A {

```
private:
    public;
```

friend class B;

class B {

```
void g(A obj) { cout << obj.a;
    cout << obj.getA();
}
```

class A
friend class B;

class B

can access members.

But A can't access B members

→ But if we want to make private member accessible by another class we have to make it friend class.

But if the other class has not declared it as friend we can't access first member in other class.

so we declare it

We can also make friend function. → that are global.
friend void display(A obj);

* Operator Overloading: → friend operator

insertion operator << < istream >> ostream <<

```
ostream & operator<< (ostream & out, const A & obj) {
    out << "a = " << obj.a << "b = " << obj.b << endl;
    return out;
}
```

```
friend ostream & operator<< (ostream & out, const A obj);
main() {
```

```
    cout << obj; << obj; << obj;
}
```

Date

```
istream & operator >> (istream & in, obj A & obj) {  
    in >> obj.A;  
    in >> obj.B;  
}
```

Static member not initialized = 0.

Size of static member is not counted in object's size.



Date _____

Header File → .h ⇒ name, Declaration

.cpp ⇒ Definition/Implementation

main ⇒ #include " .h"

Header File :

#ifndef _____h

#define _____h

Declare

#endif

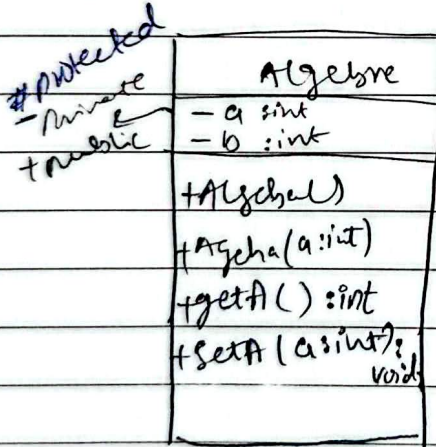
Declaration file

_____ .cpp

Implementation

Multiple file Code → create project

* UML diagrams



→ has — a

cashier — a cupine

B has — a A

* **Composition** : one class makes an object of another class in itself & that class is dependent on it.

class A {

int a;

public:

≡

{

class B

{ int b

A obj;

≡

}

int making

B obj: →

int
obj A

if new memory is allocated → composition.
Date

* **Aggregation**: No dependency of one class on the other, weak relationship (**has-a**)
Object stored as a pointer. (named as a pointer not created)
driver — engine
computer has a CPU
In composition the other class's objects' constructor is called first. → CPU computer
Destructor order is Reverse

computer
CPU

* parameterized constructor is called by member initialization.

!! C++ has a fraction lab on Thursday
[Composition → bcz new book being created.
class Book {

title, author

const
void print

~Book() {
Library {

~~Book constructor~~
Library constructor

print
print

~Library
~book

book * p; int arr;

bc arr → lib const → arr = 20

display() call print
for each book {

~delete b ~Library():

add book()

insert book size → new book

copy all elements

store the new book

delete me

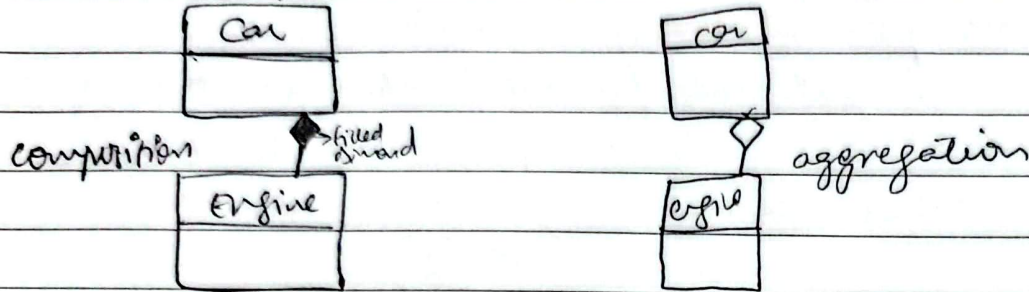
bc temp; }

Library.addbook("Book", " ")

library constructor
as soon as destructor
as function are still
called.
nameless object

Date

* UML Diagrams:



Date d = 1; $\rightarrow$ will work (Implicit Typecasting)

Date d = (Date) 1; $\rightarrow$ explicit type casting

Date d = Date(1); $\rightarrow$ gamelen to name.

* TO stop implicit type casting, add keyword "explicit" before constructor.

— ob = c1 ob(c1) —

c3 = c1 $\rightarrow$

c

A

Date

- * Inheritance - *

→ "is-a"

• rectangle is-a shape

derived
class /
child
class

↳ Base Class / parent class

→ class A;

class B: ^{derived} public A {

{ 'A' can't access any member or functions of 'B'.
'B' can't access private members of 'A'. Only functions.

→ can call A functions. (has all properties of A, & also all properties of itself)

[Cons of A
Cons of B
Des of B
Des of A] !! sequence of Constructors & Destructors !!

access modifier : 'Protected' → available within that class & also in derived classes.
members OR functions
But not outside the class.

* To call parameterized cons, we have to use member initializer list. B() : A() {};

* Derived class can access static members & functions.

* If functions have same name, latest one will be called.

* OA = OB (works on inheritance, & A only access A's part in B).

* Size of derived class equals to All member of Base class & its own class.

* If no members size is 1 byte.

Date

* Multiple Inheritance:

$A \rightarrow C \leftarrow B$

$\rightarrow C$ can have properties of A & B both.

```
class C { public A, public B
  cons A 1
  cons B
  cons C
```

* If function have same name in A & B, error will occur. We will have to use Scope Resolution Operator to call func in derived class.

* Multi-level Inheritance:

$C \rightarrow B \rightarrow A$

* Hybrid Inheritance

name
roll
name
id
marks
std::name/
cups::name

$C \rightarrow B \rightarrow A$
 $\rightarrow D$

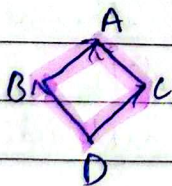
multiple & multilevel.

$C \rightarrow B \rightarrow A$
 $\rightarrow D$

name
roll
id
marks

★

* Diamond Problem:



when diff classes inherit same one class we have to use scope resolution operator as to not cause ambiguous Result.

* Java does not has diamond problem cause multiple inheritance does not exist.

Date

* Virtual Inheritance *

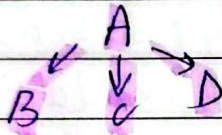
class C : virtual public person {

o to solve diamond problem

virtual pointers → classes make on their own, to identify where a Date member is coming from.
So size of class all these pointers too.

• when 1 class also called that work the other won't call it, so constructor only called once

* Hierarchy :



→ If inheritance is 'Public'

Parent Class

Inheritance Type	Public	Protected	Private
public	Public ✓	protected ✓	Private ✗
Protected	protected ✓	protected ✓	private ✗
private	private	Private	private ✗

assembler but only in Derived.

```
int sum ( char str[] ) {
```

```
    int sum = 0;
```

```
    for ( int i = 0; str[i] != '\0'; i++ ) {
```

```
        if ( str[i] >= '0' && str[i] <= '9' )
```

```
            sum += str[i] - '0';
```

```
    return sum;
```

Date

358/2

★ Polymorphism ★

child *p;

child c;

p = &c;

∴ p → {c};

child * p = new child;

Base * P = new Base;

Inheritance gives allowance to store child class address in Base class pointer;

child can't store Base class address.

Base * P = new child;

polymorphism → many forms. ^{→ pointer} pointer stays 1 → but can call & point at child classes & call its function.

- Compile time / Static polymorphism (function / overloading / operator)
- Run time / Dynamic polymorphism (Decision at Run time of function.)

Add 'virtual' with function for overloading in parent

↳ Function Overriding.

* function have pointers → to point the function.

① Virtual ptr of parent class store address of latest function of child class that the function overridden by same name.

② if function is not present in parent class, the parent pointer can't call it.

→ Destructor of parent virtual → to deallocate memory on heap of child then parent by pointing parent.

Date

- * Use 'override' keyword with the overridden function in Derived Class.
- * Dynamic Binding → show as more useful on run time.
- * Member functions → if virtual → have their own Vtable → where their address is stored, & the function called depends on the one stored in that class's Vtable.
- * pure virtual function = function prototype = 0;
Defined in Base Class → No Definition Given.
If pure virtual function exist → We can't create its object.
- * Abstract Class → No Object
↓
At least one pure virtual function.
Inherited Class has to define the function. Otherwise will also become pure virtual & object can't be created.
- * Polymorphism effective when parent class has address or reference of child object.
- * If all methods in a class are pure virtual they are called → pure Abstract or interface.
↓ keyword not available in C++.

Date

* 'final' stops overriding of function. Add where ever we want the final function to be.

* Templates:

templated functions ^{or} class

Global: template <typename T>
void Sum (T a, T b)

```
{  
    cout << a + b;  
}
```

type Determined at
Run time.

Class & Typename both
work.

```
template <typename T1, typename T2>  
void Sum (T1 a, T2 b)  
{  
  
}
```

when calling function we can add type to help implicitly.
Sum <int> (a, b);

Templated Class ^{or} class

template <typename T>

class Set {

private:

int Size;

int Elebs;

T * Data;

if Defining functions after
class we write

This before each
function or constructor.

template <class T>

Set <T>::show() { }

main;

Set <int> S1;

Set <double> S2;

Date

$B^*b = (B^*)a;$ → explicit C-style type casting

parent pointer to child ptr explicitly.

$B^*b = \text{static_cast} \langle B^* \rangle a;$

Dynamic Cast: Parent ptr to child & parent.

if parent to parent no address in child

if parent to child ~~no~~ address of child stored

$B^*b = \text{dynamic_cast} \langle B^* \rangle (a);$

* Exception Handling

TO stop Run time errors.

```
try {
```

```
}
```

```
catch ( ) {
```

```
}
```

for try-catch we have to throw some error at the try block for catch as variable.

- exception, Object, int, double, ^{string} as things we can use

- multiple catch block.

* exception → default class of C++.

what() → function of exception

Date

Jagged Array: only on heap

triangular Array for (int i=0; i<cols; i++)
 Arr[i] = new int [i+1];

Pascal Triangle

}

1
 1 1
 1 2 1
 1 3 3 1

no. of col = # of row

* To create pascal Triangle:

```
int rows = 5; int cols = 5;
int ** Arr = new int * [rows];
for (int i=0; i<rows; i++) {
  arr[i] = new int [i+1];
  arr[i][0] = 1; arr[i][i] = 1;
  for (int j=0; j<i; j++) {
    arr[i][j] =
    for (int j=1; j<i; j++) {
      arr[i][j] = arr[i-1][j] + arr[i-1][j-1];
    }
  }
}
```

short Matrix

int ** data;
 int rows, cols;

void createMatrix (Matrix &m, int r, int c);

80.125

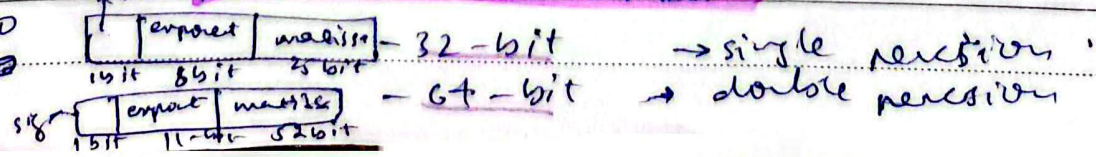
2 | 80
 2 | 40 - 0
 2 | 20 - 0
 2 | 10 - 0
 2 | 5 - 0
 2 | 4 - 1
 2 | 2 - 0
 1 | 1 - 0

1 0 1 0 0 0 0

1 0 1 0 0 0 0 . 0 0 1

0.125
 x 2
 .250 - 0
 x 2
 .500 - 0
 x 2
 1.000 - 0

IEEE 754 floating point



Date

```
int* createArray(int size) {  
    return new int[size];  
}
```

array, input, even

```
int arr[20];
```

```
for (int i=0; i<20; i++) {
```

```
    cin >> arr[i]; }
```

```
int count = 0;
```

```
for (int k=0; k<20; k++) {
```

```
    if (arr[i] % 2 == 0) {
```

```
        count++; }
```

```
int evenCount = 0; int j=0;
```

```
int* even = new int [count];
```

```
for (int i=0; i<count; i++) {
```

```
    if (arr[i] % 2 == 0) {
```

```
        even[j] = arr[i];
```

```
        j++;
```

```
    }
```

```
}
```

* file
↓ need
↓

```
for (int k=0; k<count; k++) {
```

```
    cout << even[k] << " ";
```

```
}
```

```
}
```

```
data.txt) int arr[1], count, array;
```

```
ofstream of ("data.txt");
```

version?

```
int count; int i=0; vector<int> arr;
```

```
while (i != eof) { int num;
```

```
    of << num;
```

```
    arr.push_back(num); }
```

```
int size = arr.size();
```

```
int* arr = nullptr;
```

```
int count = 0; int temp;
```

```
ifstream fin ("data.txt");
```

```
while (fin >> temp) {
```

```
    int* arr = new int [count+1];
```

```
    for (int i=0; i<count; i++) {
```

```
        newarr[i] = arr[i]; }
```

```
    newarr[count] = temp;
```

```
    delete [] arr;
```

```
    arr = newarr;
```